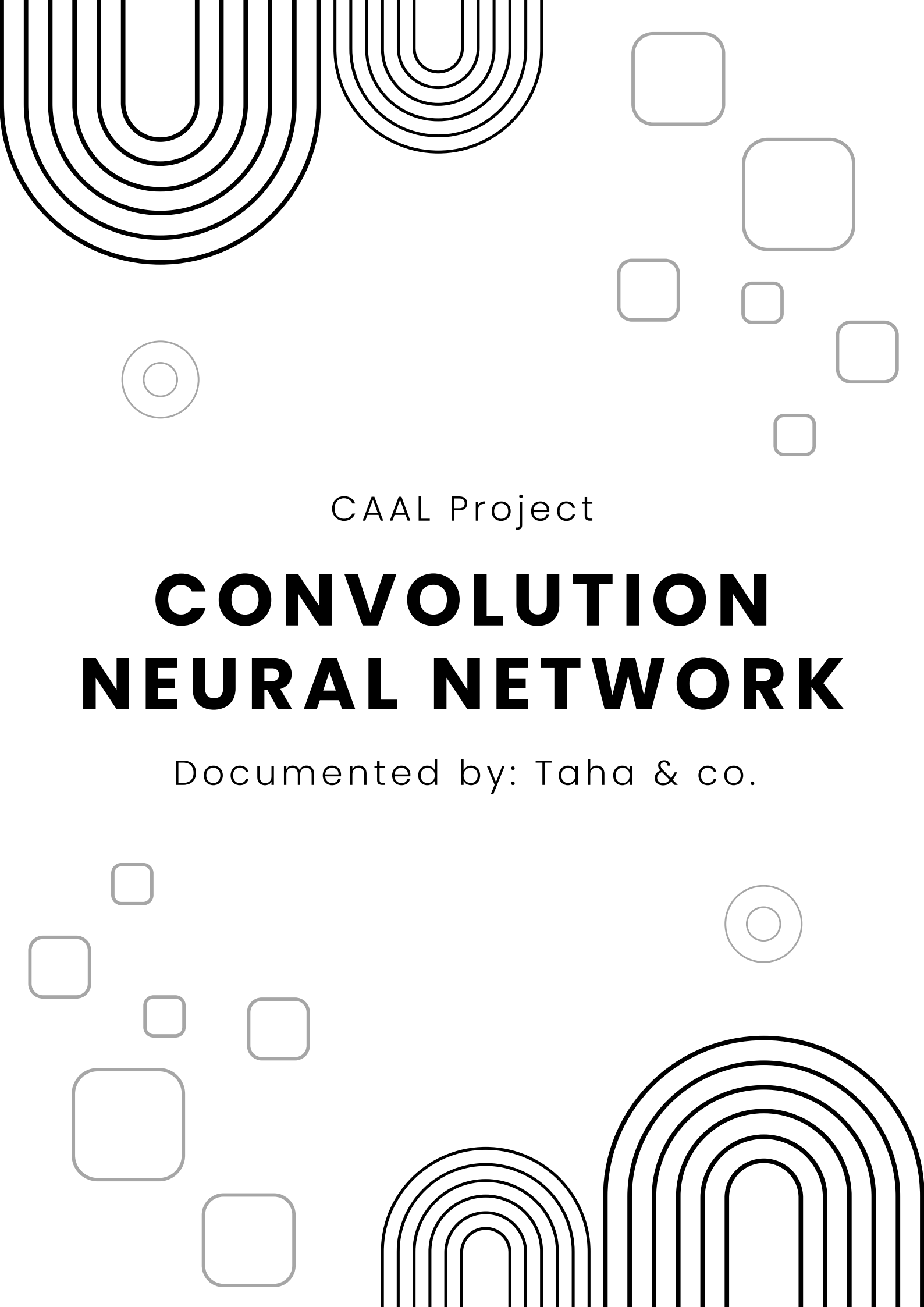




CAAL Project

CONVOLUTION NEURAL NETWORK

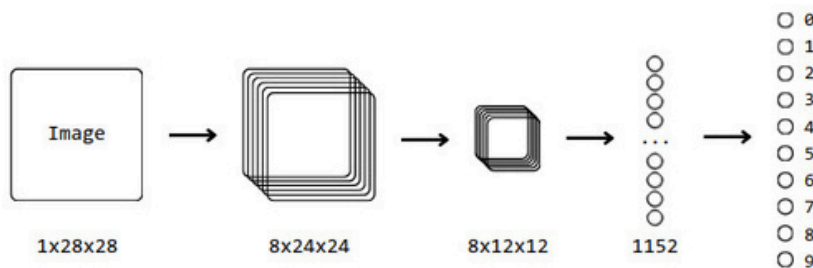
Documented by: Taha & co.

OVERVIEW

This project focuses on implementing and optimizing the inference pipeline of a Convolutional Neural Network (CNN) at the assembly level using the RISC-V Vector Extension (RVV). The primary objective is to process an input image through all the critical layers of a CNN—including convolution, pooling, activation, flattening, dense layer, and SoftMax—in a fully vectorized manner, enabling the model to generate an accurate prediction entirely on hardware. Each layer was carefully designed to work in sequence, where the output of one layer seamlessly becomes the input of the next, forming an end-to-end inference system.

By leveraging RVV, the project aimed to exploit data-level parallelism present in CNN operations and minimize the computational bottlenecks traditionally associated with scalar instruction pipelines. The system was architected such that, given the data of an input image, it would traverse the vectorized Conv2D layer and flow through Max Pooling, ReLU activation, Flatten, Dense layers and SoftMax ensuring both correctness and efficiency in the prediction phase.

METHODOLOGY AND LOGIC:



For the **Convolutional Layer (Conv2D)**, the approach focuses on vectorizing the core dot product operation between the input image patch and the convolution filter. By loading the image patch and filter row into vector registers and using the fused multiply-accumulate instruction (*vfmacc.vv*), the entire row-wise multiplication and accumulation was done in a vectorized manner. Subsequent reduction of the result vector was achieved using *vfredsum.vs*, followed by bias addition and storage of the scalar result. This reduced the overhead of manually looping through each element and allowed simultaneous processing of multiple filter coefficients and input pixels, significantly enhancing the throughput.

The **ReLU activation** function was implemented using full-array vectorization. The vector registers were initialized with zero using **vfmv.v.f**, and each input vector was compared elementwise using **vfmmax.vv** to ensure all negative values were replaced by zero. This method avoided any explicit branching and took advantage of RVV's ability to perform these operations across wide vectors, ensuring that activation functions, despite being simple, did not become a computational bottleneck.

The **Max Pooling** function uses RVV instructions to perform 2x2 pooling over multi-channel inputs. The key operation is **vlseg2e32.v**, which is essential as it loads two rows of the 2x2 window simultaneously by fetching segments of two 32-bit elements with a configurable stride. This effectively captures the desired 2x2 matrix directly from memory. The loaded segments are processed using **vfmmax.vv** and reduced to a single maximum value with **vfredmax.vs**. This result is then stored in the output feature map. The approach optimizes data access and leverages vectorization to efficiently implement max pooling.

For the **Flatten layer**, the vectorization was particularly optimized by using stride vector loads (**vlse32.v**), which allowed the loading of all channel values corresponding to a specific (*i, j*) spatial location across all channels in a single instruction. This approach eliminated the need for nested loops over channels, reducing the flattening operation to a simple sequence of vector load and store instructions. The gathered vectors were stored linearly in the output buffer, ensuring that the data was efficiently prepared for the subsequent dense layer.

In the **Dense layer**, the dot product between the input activations and the weight matrix was vectorized using the same fused multiply-accumulate pattern (**vfmacc.vv**). The weight vectors were loaded per output neuron, and the input was processed in blocks corresponding to the vector length, allowing the computation of each output neuron to be handled efficiently. This greatly reduced the per-element operation overhead common in scalar implementations, leading to faster inference times.



The core functionality in **SoftMax** relies on computing the exponential of each element in the array. Initially, we used an **exp_x** function that took an integer input and raised Euler's number to that power. While this approximation gave us about 98% accuracy, it came at a structural cost—the code was inherently scalar and non-vectorizable. The old version looked like this in logic:

```
exp_x:  
  Convert float to int  
  Multiply EULER_NUMBER by itself in a loop  
  If x was negative, return reciprocal
```

This meant that we had conditional branches and a loop that only operated on one value at a time, making it fundamentally unfit for vector execution. By replacing it with a Taylor series approximation, the entire structure of the SoftMax routine transformed:

```
exp_taylor  
  Initialize result and term to 1.0  
  Repeat:  
    Multiply term by (x / i)  
    Add term to result
```

This approach maps cleanly to vector operations, **vfmul.vv**, **vfadd.vv**, and **vfdiv.vv**, and allowed us to calculate $\exp(x)$ in parallel across multiple values.



CHALLENGES AND RESOLUTIONS:

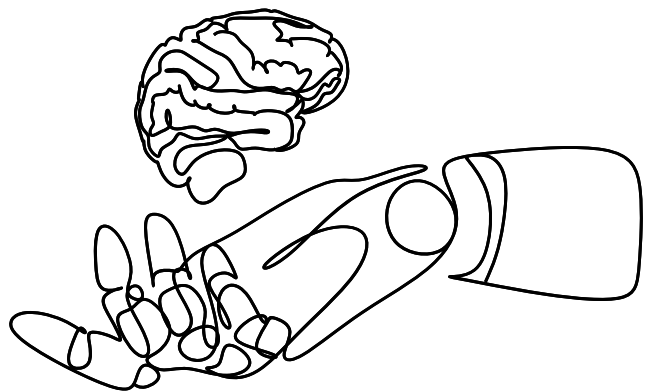
For the **Convolutional Layer**, the initial hurdle was recognizing opportunities for vectorization in what was originally a highly scalar and deeply nested loop structure. To address this, simpler vector operations such as dot product and matrix-vector multiplication were first implemented separately to build understanding. This exploration revealed that the convolution inner loops, when reorganized, naturally exposed patterns amenable to vectorization. By removing unnecessary loops (like those over channels, fixed to 1 in our case) and restructuring the inner computations as direct vector dot products, the convolution was successfully vectorized using **vle32.v** and **vfmac.vv**. Custom debugging scripts were also created to ease the validation process by converting RVV register outputs directly into floats.

In the **Max Pooling Layer**, the irregular and non-contiguous memory accesses posed a challenge, as pooling windows at edges could not be efficiently vectorized. The solution was to selectively vectorize the inner kernel computations while keeping the control flow and address calculations in scalar, ensuring correctness without sacrificing too much performance.

For the **Flatten Layer**, the key challenge was maintaining the correct data order while flattening the 3D feature map. Getting the offsets right to map channel-wise data into a linear buffer was initially tricky and prone to indexing errors. This was efficiently resolved by using **vlse32.v** for stride loads, simplifying the process.

For the **ReLU Layer**, the implementation was relatively straightforward. The main task was ensuring clean vectorized comparison across the entire input using **vfmax.vv**, while handling edge cases for inputs not aligned with the vector length.

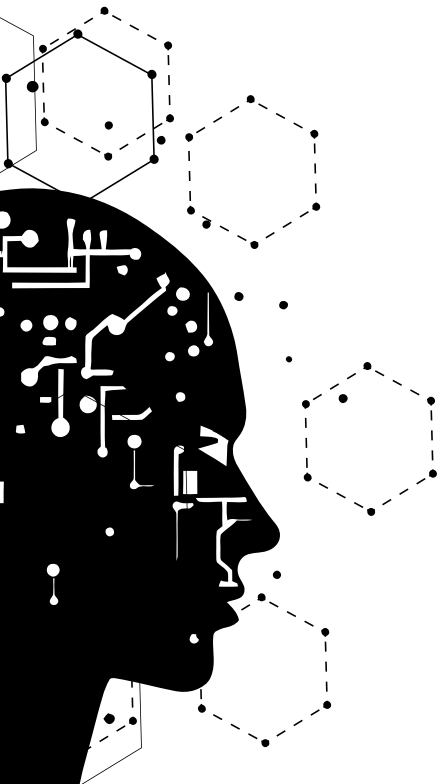
The **Dense Layer** faced issues with precise pointer calculations during matrix-vector operations and integrating bias addition efficiently. These were resolved by carefully combining the dot product and bias addition into a single loop using **vfmac.vv**, minimizing memory passes and ensuring correctness.



The main bottleneck in vectorizing **SoftMax** was the ***exp_x*** function. Because it relied on integer casting and sequential multiplication with conditional branches, it fundamentally couldn't be expressed with RVV instructions in a clean way. We tried various tricks—loop unrolling, masking, and condition flattening—but nothing really worked. Switching to the Taylor series model for ***exp(x)*** changed everything. The loop became uniform and arithmetic-only. No branches, no casts, just math. That allowed us to rewrite the core SoftMax routine as:

```
softmax:
    Compute the exponential of each element,
    vfmul.vv,   vfadd.vv, vfdiv.vv
    vfredosum.vv to get sum
    Divide by vfdiv.vv
```

AND WE ARE DONE!



Syed Taha
Hamna Sajid
Hadiya Muneeb
Ammar Bin Amir Khan